Laporan Proyek Berpikir Komputasional (WI1102)

Kelas K05



Disusun Oleh:

Jovan Nathaniel Permana (16026295)
Qamila Annur Firdausi (16026380)
Abira Mutia Salsabila (16026060)
Ezra Hadyan Wicaksono (16026055)

Fakultas FMIPA Institut Teknologi Bandung

Semester 1 Tahun 2026/2027

1 Dasar-dasar Berpikir Komputasional yang Digunakan

1.1 Algoritma Newton-Raphson

1.1.1 Dekomposisi

Algoritma Newton-Raphson sebenarnya adalah algoritma optimasi yang bersifat iteratif, dengan tujuan utama untuk mencari akar-akar/titik potong sebuah fungsi $f(x)$ dengan sumbu x. Dalam proyek ini, algoritma Newton-Raphson difokuskan untuk mencari akar-akar dari sebuah polinom dengan derajat tertentu, sesuai masukan dari pengguna (*user*). Melalui prinsip dekomposisi (pemecahan suatu masalah dengan skala besar menjadi skala yang lebih kecil), algoritma Newton-Raphson untuk mencari akar-akar sebuah fungsi polinom dapat dipecah menjadi beberapa tugas berikut.

1. Menginput derajat polinom (N), nilai koefisien-koefisiennya, serta nilai tebakan awal (x_0) dari user.
2. Untuk nilai tebakan awal $x = x_0$ yang diberikan, hitung nilai fungsi $f(x_0)$ beserta turunannya, yaitu $f'(x_0)$.
3. Nilai x diperbarui secara terus-menerus melalui prosedur iteratif dengan persamaan
$$x_{n+1} \leftarrow x_n - \frac{f(x_n)}{f'(x_n)} \tag{1}$$
hingga nilai *error* yang terjadi dalam penentuan akar menjadi tidak signifikan atau berada di bawah ambang batas tertentu.
4. Jika $f'(x) = 0$ pada suatu tahap tertentu atau perulangan pada langkah 3 tidak dapat selesai (*infinite loop*), maka proses perhitungan akar gagal dilakukan.

1.1.2 Pengenalan Pola

Algoritma Newton-Raphson memiliki pola yang mirip dengan algoritma optimasi, yaitu algoritma yang menekankan perhitungan matematika secara iteratif (berulang) untuk mencapai suatu tujuan. Dalam algoritma Newton-Raphson, persamaan (1) dijalankan secara berulang-ulang untuk mencapai suatu tujuan, yaitu akar persamaan polinom. Selama perulangan tersebut, nilai x akan terus diperbarui (*update*) hingga error dalam pencarian akar menjadi tidak signifikan atau lebih kecil dari ambang batas tertentu.

1.1.3 Abstraksi

Algoritma Newton-Raphson membutuhkan beberapa informasi penting, yaitu:

1. Derajat polinom N.
2. Koefisien-koefisien pada polinom.
3. Nilai tebakan awal x_0.

Dalam proyek ini, derajat polinom N disimpan di dalam variabel bertipe `int`, sementara nilai tebakan awal x_0 disimpan di dalam variabel bertipe `float`. Nilai-nilai koefisien pada polinom disimpan di dalam sebuah array `float` yang berukuran $N + 1$.

1.2 Algoritma Pinjam Buku

1.2.1 Dekomposisi

1.2.2 Pengenalan Pola

1.2.3 Abstraksi

1.3 Algoritma Labirin

1.3.1 Dekomposisi

1.3.2 Pengenalan Pola

1.3.3 Abstraksi

2 Lampiran

2.1 Lampiran 1: Kode Python Tanpa Menggunakan Subprogram

2.1.1 Algoritma Newton-Raphson

Kode 1: Kode Python algoritma Newton-Raphson tanpa subprogram

```
1 import time
2 """ Newton Raphson (tanpa subprogram) """
3
4 # program utama
5 # input
```

```
N = int(input('Masukkan derajat polinomial: '))
N += 1
coefs = [0]*N
for i in range(N):
    if(i == 0):
        val = float(input('Masukkan nilai konstanta: '))
    else:
        val = float(input(f'Masukkan nilai koefisien x^{i}: '))

    coefs[i] = val

x = float(input('Masukkan tebakan awal x: '))
epsilon = 1e-4
k = 0
max_iter = 1e6
# proses iterasi
start_time = time.perf_counter()
while True:
    k += 1
    val = 0.0
    el = 0
    element = 1
    while el <= (N-1):
        val += coefs[el] * (x ** el)
        el += 1
    grad = 0.0
    while element <= (N-1):
        grad += element * coefs[element] * (x ** (element - 1))
        element += 1

    if grad == 0:
        print('Newton-Raphson gagal: turunan = 0')
        break

    x_new = x - (val / grad)
    print(f'Iterasi ke-{k} --> Nilai x: {x_new:.6f} Gradien: {grad:.6f}')
    delta = x_new - x
    if delta < 0:
        delta = -delta

```

```
    if val > 0:
        absval = val
    else:
        absval = -val

    if (delta < epsilon) and (absval < epsilon):
        print(f'Akar ditemukan: {x_new:.6f}')
        print(f'Jumlah iterasi: {k}')
        break
    elif k >= max_iter:
        print(f'\nGAGAL: Akar tidak ditemukan setelah {k} iterasi.')
        break

    x = x_new

end_time = time.perf_counter()
execution_time = end_time - start_time
print(f"Waktu eksekusi: {execution_time:.6f} detik")
```

2.1.2 Algoritma Pinjam Buku

Kode 2: Kode Python algoritma Pinjam Buku tanpa subprogram

```
import time
"""Pinjam Buku (Tanpa Subprogram)"""
```

2.1.3 Algoritma Labirin

Kode 3: Kode Python algoritma Labirin tanpa subprogram

```
import time
"""labirin (tanpa Subprogram)"""
# input
N = int(input("Masukkan ukuran labirin: "))
matrix = []
for i in range(N):
    col = [0]*N
    matrix += [col]

for y in range(N):
    for x in range(N):
```

```
12          val = int(input(f'Apakah koordinat ({x},{y})
                bisa dilewati (1) atau tidak (0): '))
13          matrix[y][x] = val
14      print('================================')
15
16  # proses
17  start_time = time.perf_counter()
18  # inisialisasi pointer dan arahnya
19  status_sisi = [[-1,-1], [-1,-1], [-1,-1], [-1,-1]]
20  # cari titik awal di sisi atas
21  for i in range(N):
22      if matrix[0][i] == 1:
23          status_sisi[0] = [i,0]
24          break
25  # cari titik awal di sisi bawah
26  for i in range(N):
27      if matrix[N-1][i] == 1:
28          status_sisi[1] = [i,N-1]
29          break
30  # cari titik awal di sisi kiri
31  for i in range(N):
32      if matrix[i][0] == 1:
33          status_sisi[2] = [0,i]
34          break
35  # cari titik awal di sisi kanan
36  for i in range(N):
37      if matrix[i][N-1] == 1:
38          status_sisi[3] = [N-1, i]
39          break
40
41  # tentukan endpoint
42  endpoints = []
43  for i in range(4):
44      if status_sisi[i] != [-1,-1]:
45          endpoints += [status_sisi[i]]
46
47  # tentukan titik mulai
48  pointer = endpoints[0]
49  direction = [0,0,0,0]
50  avail_move = [0,0,0,0]
51  forbidden_move = [0,0,0,0]
52  clockwise = True
53  change_clockwise = 0
```

```
# jalankan iterasi
while pointer != endpoints[1]:
    change_clockwise += 1
    print(f'Titik x: {pointer[0]}, y: {pointer[1]}
       berhasil dilewati')
    # cek arah yang tersedia
    if (pointer[0] == 0) and (pointer[1] == 0):
        direction = [0,1,1,0]
    elif (pointer[0] == 0) and (pointer[1] == N-1):
        direction = [1,1,0,0]
    elif (pointer[0] == (N-1)) and (pointer[1] == 0):
        direction = [0,0,1,1]
    elif (pointer[0] == (N-1)) and (pointer[1] == (N-1))
       :
        direction = [1,0,0,1]
    elif (0 < pointer[0] < (N-1)) and (pointer[1] == 0):
        direction = [0,1,1,1]
    elif (0 < pointer[0] < (N-1)) and (pointer[1] == (N
       -1)):
        direction = [1,1,0,1]
    elif (pointer[0] == 0) and (0 < pointer[1] < (N-1)):
        direction = [1,1,1,0]
    elif (pointer[0] == (N-1)) and (0 < pointer[1] < (N
       -1)):
        direction = [1,0,1,1]
    else:
        direction = [1,1,1,1]

    # cek apakah bisa dilewati
    i = 0
    while(i < 4):
        if direction[i] == 1:
            if (i == 0) and (forbidden_move[i] == 0):
                bukan_dinding = int(matrix[pointer
                   [1]-1][pointer[0]] == 1)
                avail_move[i] = bukan_dinding
            elif (i == 1) and (forbidden_move[i] == 0):
                bukan_dinding = int(matrix[pointer[1]][
                   pointer[0]+1] == 1)
                avail_move[i] = bukan_dinding
            elif (i == 2) and (forbidden_move[i] == 0):
                bukan_dinding = int(matrix[pointer
                   [1]+1][pointer[0]] == 1)
```

```
90                  avail_move[i] = bukan_dinding
91              elif (i == 3) and (forbidden_move[i] == 0):
92                  bukan_dinding = int(matrix[pointer[1]][
                        pointer[0]-1] == 1)
93                  avail_move[i] = bukan_dinding
94          else:
95              avail_move[i] = 0
96          i += 1
97      # Jika jalan buntu
98      if avail_move == [0,0,0,0]:
99          i = 0
100         while(i < 4):
101             if direction[i] == 1:
102                 if i == 0:
103                     bukan_dinding = int(matrix[pointer
                            [1]-1][pointer[0]] == 1)
104                     avail_move[i] = bukan_dinding
105                 elif i == 1:
106                     bukan_dinding = int(matrix[pointer
                            [1]][pointer[0]+1] == 1)
107                     avail_move[i] = bukan_dinding
108                 elif i == 2:
109                     bukan_dinding = int(matrix[pointer
                            [1]+1][pointer[0]] == 1)
110                     avail_move[i] = bukan_dinding
111                 else:
112                     bukan_dinding = int(matrix[pointer
                            [1]][pointer[0]-1] == 1)
113                     avail_move[i] = bukan_dinding
114             else:
115                 avail_move[i] = 0
116             i += 1
117
118     # Pilih jalur
119     if clockwise:
120         j = 0
121         while (j < 4):
122             if avail_move[j] == 1:
123                 if j == 0:
124                     pointer[1] -= 1
125                     forbidden_move = [0,0,1,0]
126                     break
127                 elif j == 1:
```

```
128                         pointer[0] += 1
129                         forbidden_move = [0,0,0,1]
130                         break
131                     elif j == 2:
132                         pointer[1] += 1
133                         forbidden_move = [1,0,0,0]
134                         break
135                     else:
136                         pointer[0] -= 1
137                         forbidden_move = [0,1,0,0]
138                         break
139                 j += 1
140         else:
141             j = 3
142             while (j >= 0):
143                 if avail_move[j] == 1:
144                     if j == 0:
145                         pointer[1] -= 1
146                         forbidden_move = [0,0,1,0]
147                         break
148                     elif j == 1:
149                         pointer[0] += 1
150                         forbidden_move = [0,0,0,1]
151                         break
152                     elif j == 2:
153                         pointer[1] += 1
154                         forbidden_move = [1,0,0,0]
155                         break
156                     else:
157                         pointer[0] -= 1
158                         forbidden_move = [0,1,0,0]
159                         break
160                 j -= 1
161 
162         if (change_clockwise % 3) == 0:
163             clockwise = not clockwise
164 
165 print(f'Titik x: {pointer[0]}, y: {pointer[1]} berhasil
        dilewati')
166 end_time = time.perf_counter()
167 execution_time = end_time - start_time
168 print('Labirin Diselesaikan!')
169 print(f"Waktu eksekusi: {execution_time:.6f} detik")
```

2.2 Lampiran 2: Kode Python Dengan Menggunakan Subprogram

2.2.1 Algoritma Newton-Raphson

Kode 4: Kode Python algoritma Newton-Raphson dengan subprogram

```python
import time
""" Newton Raphson (dengan subprogram) """

# definisi fungsi
def function(x, coefs):
    value = 0.0
    for i in range(len(coefs)):
        value += coefs[i] * (x**i)
    return value

def gradient(x, coefs):
    value = 0.0
    for i in range(1, len(coefs)):
        value += i * coefs[i] * (x**(i - 1))
    return value

def absol(x):
    value = (x >= 0) * x + (x < 0) * (-x)
    return value

# program utama
# input
N = int(input('Masukkan derajat polinomial: '))
N += 1
coefs = [0]*N
for i in range(N):
    if(i == 0):
        val = float(input('Masukkan nilai konstanta: '))
    else:
        val = float(input(f'Masukkan nilai koefisien x^{
            i}: '))

    coefs[i] = val

x = float(input('Masukkan tebakan awal x: '))
epsilon = 1e-4
k = 0
```

```
max_iter = 1e6
# proses iterasi
start_time = time.perf_counter()
while True:
    k += 1
    val = function(x, coefs)
    grad = gradient(x, coefs)
    if grad == 0:
        print('Newton-Raphson gagal: turunan = 0')
        break

    x_new = x - (val / grad)
    print(f'Iterasi ke-{k} --> Nilai x: {x_new:.6f}
       Gradien: {grad:.6f}')

    if (absol(x_new - x) < epsilon) and (absol(val) <
       epsilon):
        print(f'Akar ditemukan: {x_new:.6f}')
        print(f'Jumlah iterasi: {k}')
        break
    elif k >= max_iter:
        print(f'\nGAGAL: Akar tidak ditemukan setelah {k
           } iterasi.')
        break

    x = x_new

end_time = time.perf_counter()
execution_time = end_time - start_time
print(f"Waktu eksekusi: {execution_time:.6f} detik")
```

2.2.2 Algoritma Pinjam Buku

Kode 5: Kode Python algoritma Pinjam Buku dengan subprogram

```
import time
"""Pinjam Buku (dengan Subprogram)"""
```

2.2.3 Algoritma Labirin

Kode 6: Kode Python algoritma Labirin dengan subprogram

```python
import time
"""labirin (dengan Subprogram)"""
# definisi fungsi
def choose_dir(pointer):
    if (pointer[0] == 0) and (pointer[1] == 0):
        direction = [0,1,1,0]
    elif (pointer[0] == 0) and (pointer[1] == N-1):
        direction = [1,1,0,0]
    elif (pointer[0] == (N-1)) and (pointer[1] == 0):
        direction = [0,0,1,1]
    elif (pointer[0] == (N-1)) and (pointer[1] == (N-1))
       :
        direction = [1,0,0,1]
    elif (0 < pointer[0] < (N-1)) and (pointer[1] == 0):
        direction = [0,1,1,1]
    elif (0 < pointer[0] < (N-1)) and (pointer[1] == (N
       -1)):
        direction = [1,1,0,1]
    elif (pointer[0] == 0) and (0 < pointer[1] < (N-1)):
        direction = [1,1,1,0]
    elif (pointer[0] == (N-1)) and (0 < pointer[1] < (N
       -1)):
        direction = [1,0,1,1]
    else:
        direction = [1,1,1,1]
    return direction
# program utama
# input
N = int(input("Masukkan ukuran labirin: "))
matrix = []
for i in range(N):
    col = [0]*N
    matrix += [col]

for y in range(N):
    for x in range(N):
        val = int(input(f'Apakah koordinat ({x},{y})
           bisa dilewati (1) atau tidak (0): '))
        matrix[y][x] = val
    print('================================')

# proses
start_time = time.perf_counter()
```

```
# inisialisasi pointer dan arahnya
status_sisi = [[-1,-1], [-1,-1], [-1,-1], [-1,-1]]
# cari titik awal di sisi atas
for i in range(N):
    if matrix[0][i] == 1:
        status_sisi[0] = [i,0]
        break
# cari titik awal di sisi bawah
for i in range(N):
    if matrix[N-1][i] == 1:
        status_sisi[1] = [i,N-1]
        break
# cari titik awal di sisi kiri
for i in range(N):
    if matrix[i][0] == 1:
        status_sisi[2] = [0,i]
        break
# cari titik awal di sisi kanan
for i in range(N):
    if matrix[i][N-1] == 1:
        status_sisi[3] = [N-1, i]
        break

# tentukan endpoint
endpoints = []
for i in range(4):
    if status_sisi[i] != [-1,-1]:
        endpoints += [status_sisi[i]]

# tentukan titik mulai
pointer = endpoints[0]
direction = [0,0,0,0]
avail_move = [0,0,0,0]
forbidden_move = [0,0,0,0]
clockwise = True
change_clockwise = 0
# jalankan iterasi
while pointer != endpoints[1]:
    change_clockwise += 1
    print(f'Titik x: {pointer[0]}, y: {pointer[1]}
       berhasil dilewati')
    # cek arah yang tersedia
    direction = choose_dir(pointer)
```

```
    # cek apakah bisa dilewati
    i = 0
    while(i < 4):
        if direction[i] == 1:
            if (i == 0) and (forbidden_move[i] == 0):
                bukan_dinding = int(matrix[pointer
                    [1]-1][pointer[0]] == 1)
                avail_move[i] = bukan_dinding
            elif (i == 1) and (forbidden_move[i] == 0):
                bukan_dinding = int(matrix[pointer[1]][
                    pointer[0]+1] == 1)
                avail_move[i] = bukan_dinding
            elif (i == 2) and (forbidden_move[i] == 0):
                bukan_dinding = int(matrix[pointer
                    [1]+1][pointer[0]] == 1)
                avail_move[i] = bukan_dinding
            elif (i == 3) and (forbidden_move[i] == 0):
                bukan_dinding = int(matrix[pointer[1]][
                    pointer[0]-1] == 1)
                avail_move[i] = bukan_dinding
        else:
            avail_move[i] = 0
        i += 1
    # Jika jalan buntu
    if avail_move == [0,0,0,0]:
        i = 0
        while(i < 4):
            if direction[i] == 1:
                if i == 0:
                    bukan_dinding = int(matrix[pointer
                        [1]-1][pointer[0]] == 1)
                    avail_move[i] = bukan_dinding
                elif i == 1:
                    bukan_dinding = int(matrix[pointer
                        [1]][pointer[0]+1] == 1)
                    avail_move[i] = bukan_dinding
                elif i == 2:
                    bukan_dinding = int(matrix[pointer
                        [1]+1][pointer[0]] == 1)
                    avail_move[i] = bukan_dinding
                else:
                    bukan_dinding = int(matrix[pointer
                        [1]][pointer[0]-1] == 1)
```

```
                    avail_move[i] = bukan_dinding
            else:
                avail_move[i] = 0
            i += 1

    # Pilih jalur
    if clockwise:
        j = 0
        while (j < 4):
            if avail_move[j] == 1:
                if j == 0:
                    pointer[1] -= 1
                    forbidden_move = [0,0,1,0]
                    break
                elif j == 1:
                    pointer[0] += 1
                    forbidden_move = [0,0,0,1]
                    break
                elif j == 2:
                    pointer[1] += 1
                    forbidden_move = [1,0,0,0]
                    break
                else:
                    pointer[0] -= 1
                    forbidden_move = [0,1,0,0]
                    break
            j += 1
    else:
        j = 3
        while (j >= 0):
            if avail_move[j] == 1:
                if j == 0:
                    pointer[1] -= 1
                    forbidden_move = [0,0,1,0]
                    break
                elif j == 1:
                    pointer[0] += 1
                    forbidden_move = [0,0,0,1]
                    break
                elif j == 2:
                    pointer[1] += 1
                    forbidden_move = [1,0,0,0]
                    break
```

```
                else:
                    pointer[0] -= 1
                    forbidden_move = [0,1,0,0]
                    break
            j -= 1

    if (change_clockwise % 3) == 0:
        clockwise = not clockwise

print(f'Titik x: {pointer[0]}, y: {pointer[1]} berhasil
    dilewati')
end_time = time.perf_counter()
execution_time = end_time - start_time
print('Labirin Diselesaikan!')
print(f"Waktu eksekusi: {execution_time:.6f} detik")
```